

AGÊNCIA DE VOOS



PUC Minas

DISCIPLINA

AED III

ETAPA

Fase II - Estruturas de indexação e ordenação externa

INTEGRANTES

Luiz Henrique de Paula Diniz

Eder Arthur

REPOSITÓRIO DO PROJETO

github.com/LuizDiniz85306/AgenciaDeVoos

Apresentação

Este documento registra as decisões de projeto e as evidências de implementação da Fase II da aplicação Agência de Voos. O foco desta etapa é manter os dados íntegros, localizáveis e ordenáveis em memória secundária, inclusive após a reinicialização da aplicação.

Objetivos da Fase II

Entrega	Aplicação no projeto
CRUD e relacionamento 1:N	Cliente é o lado 1 e Voo é o lado N; a FK idCliente é armazenada no próprio voo.
Hash Extensível	Busca direta de Cliente e Voo pela PK, com diretório e buckets gravados em arquivos binários.
Árvore B+	Consulta por idCliente e recuperação de todos os IDs de voos associados à FK.
Ordenação externa	Geração de runs e intercalação em disco por data/hora, preservando o arquivo principal.

Organização do documento

Seção	Conteúdo
1. Modelo de dados	Entidades, chave estrangeira e regras de integridade referencial.
2. Arquitetura	Responsabilidades das telas, serviços, entidades e arquivos binários.
3. Hash Extensível	Persistência do índice de chave primária e manutenção no CRUD.
4. Árvore B+	Estrutura de consulta por FK e resposta para ocorrências repetidas.
5. Ordenação externa	Runs, intercalação balanceada e arquivo final ordenado.
6. Integridade	Sincronização entre dados e índices e justificativas das decisões.

Observação de escopo. A associação legada VooCliente era N:N. Nesta fase, ela foi consolidada em Voo.idCliente para atender ao relacionamento 1:N solicitado. Uma migração única preserva a primeira associação ativa de cada voo sem apagar o arquivo antigo.

1. Modelo de dados e relacionamento

As entidades persistidas nesta fase são Cliente e Voo. A chave estrangeira foi posicionada no lado N: cada voo armazena idCliente. Assim, um cliente pode aparecer em diversos voos sem a necessidade de tabela intermediária.



Um cliente pode ter vários voos; cada voo referencia no máximo um cliente.

Regras de integridade referencial

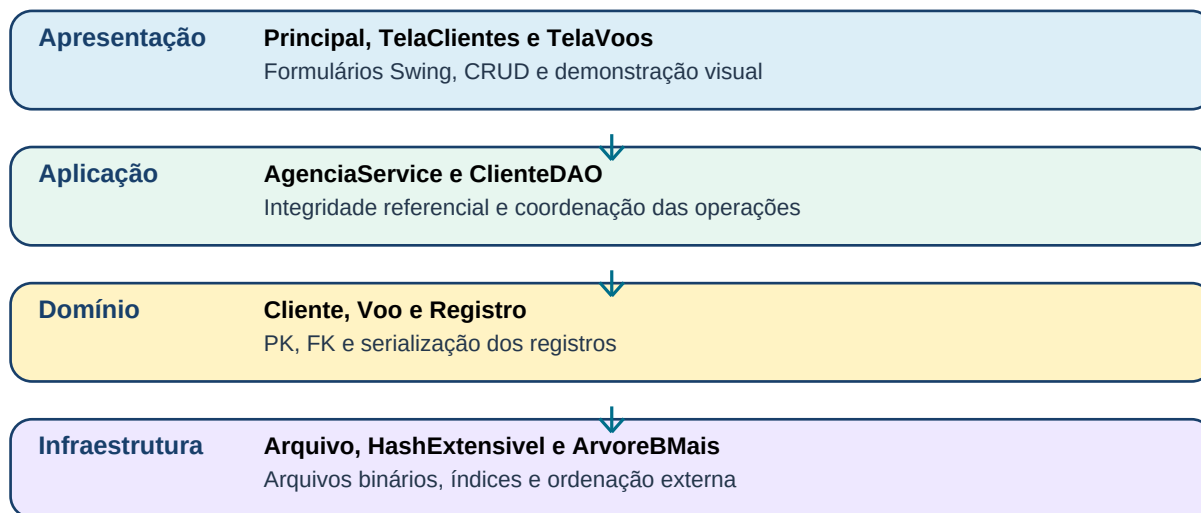
Regra	Garantia implementada
FK válida	O serviço aceita -1 para voo não associado ou um ID positivo existente no arquivo de clientes.
Cliente com voos	A interface bloqueia a exclusão do cliente quando a B+ indica voos associados.
Busca dos associados	A consulta por cliente usa a B+ para obter todos os IDs e confirma a FK no voo recuperado.
Atualização da associação	O ID deixa a chave anterior e passa a compor a nova chave da B+.

Decisão 1

O uso de idCliente diretamente em Voo materializa o lado N do relacionamento e simplifica a persistência. A regra também é aplicada dentro de AgenciaService; dessa forma, um chamador que não use a interface não consegue criar um voo com uma FK inexistente.

2. Arquitetura proposta

A solução foi organizada por responsabilidades. As telas apresentam e validam os dados. Os serviços coordenam o CRUD de voos, a integridade do relacionamento e a recuperação de índice. O componente genérico Arquivo cuida da persistência binária e do Hash; a Árvore B+ atende especificamente a consulta pelo cliente.

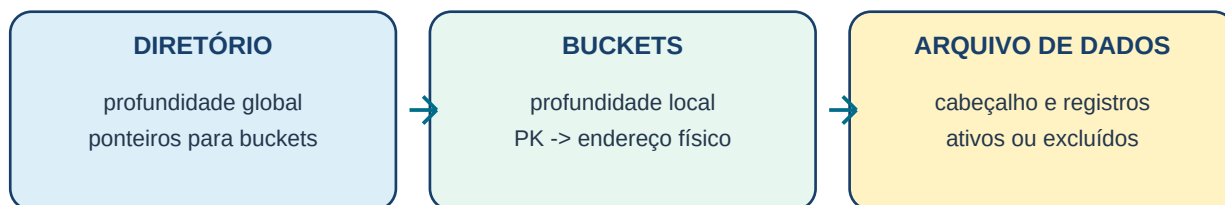


Responsabilidades principais

Componente	Responsabilidade
Principal	Inicializa a aplicação e conecta as telas no mesmo fluxo de navegação.
TelaClientes e TelaVoos	Permitem CRUD, seleção de cliente, consulta de voos, associação e ordenação.
AgenciaService	Coordena voos, valida FK, consulta B+ e recupera o índice após falhas.
Arquivo	Grava registros binários, mantém espaços livres e atualiza o Hash da PK.
HashExtensivel e ArvoreBMais	Persistem os índices de PK e de FK em arquivos secundários.

3. Hash Extensível persistente

O Hash Extensível é a estrutura de acesso direto utilizada por Arquivo. Ele mapeia a chave primária para o endereço físico em que o registro está gravado. Como o índice é persistido, a busca por ID continua disponível depois que a aplicação é fechada e reaberta.



Busca direta: a PK seleciona o bucket, e o bucket aponta para o endereço físico do registro.

Arquivos e atualização

Arquivo ou operação	Conteúdo e comportamento
*.hash.dir.db	Profundidade global e diretório de ponteiros para buckets.
*.hash.buckets.db	Profundidade local e pares PK -> endereço físico.
Inclusão	A nova PK e o endereço físico são inseridos; buckets se dividem quando necessário.
Atualização e exclusão	O endereço é atualizado quando o registro muda; exclusões removem a chave do índice.

Decisão 2 - recuperação do Hash

Na abertura, o Hash é carregado dos seus arquivos binários. Uma conferência compara os registros ativos e seus endereços com o índice; a reconstrução por varredura ocorre apenas se os arquivos estiverem ausentes ou se houver inconsistência. Assim, a persistência é usada de fato e há uma rota segura de recuperação.

4. Árvore B+ e consulta pela FK

A Árvore B+ secundária é mantida no arquivo voos.id_cliente.bplus.db. As chaves ficam nas folhas e cada chave idCliente aponta para uma lista ordenada dos IDs de voo. Isso permite recuperar vários registros para a mesma FK sem percorrer todo o arquivo principal.



As folhas mantêm a FK e todos os IDs associados; cada ID é recuperado no arquivo de voos pelo Hash.

Caminho da consulta 1:N

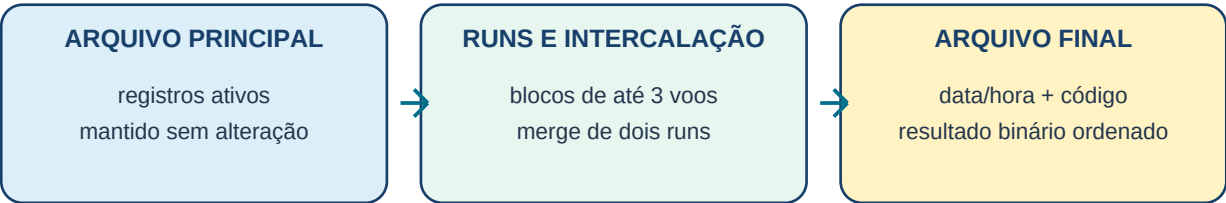
Etapa	Processamento
1. Informar cliente	A tela recebe o ID do cliente ou seleciona uma linha da lista de clientes cadastrados.
2. Consultar B+	A busca localiza a folha de idCliente e devolve todos os IDs de voo daquela chave.
3. Ler os voos	Cada ID retornado é lido pelo Hash Extensível do arquivo de voos.
4. Exibir resultado	A interface apresenta os voos associados no painel Voos do Cliente.

Decisões 3 e 4

Uma FK repetida acrescenta outro ID na lista de valores da mesma chave, sem duplicar o cliente. A serialização armazena nós, filhos, valores das folhas e seu encadeamento. Ao reiniciar, a estrutura é recarregada e conferida com os voos ativos.

5. Ordenação externa por intercalação

A ordenação externa usa a chave composta data/hora do voo e código como desempate. Em vez de carregar todos os registros em memória principal, a rotina gera pequenos runs ordenados em disco e os intercala dois a dois até produzir um único arquivo final.



O resultado é voos_ordenados_por_data_hora.db; os arquivos temporários são removidos após a intercalação.

Decisão 5 - runs e intercalação

O bloco de trabalho contém até três voos. Cada bloco é ordenado em memória e gravado como um run temporário. Nas passagens seguintes, dois runs são mesclados em outro arquivo temporário. Ao final, todos os voos ativos aparecem no arquivo de saída, em ordem cronológica, sem alterar a ordem física do arquivo principal.

Entrada	Saída
Arquivo de voos com registros ativos	Arquivo binário ordenado por data/hora, com código como desempate.
Comparador: data/hora e código	Runs temporários e intercalações realizados em memória secundária.

6. Sincronização, recuperação e decisões

A manutenção dos índices acompanha os dados em todas as operações. Por esse motivo, as estruturas são atualizadas no fluxo de inclusão, alteração e exclusão. Há verificações na abertura e uma recuperação da B+ a partir do arquivo de voos quando alguma etapa do CRUD falha.

Operação	Dados e índices
Criar voo	Arquivo grava o voo e atualiza o Hash; AgenciaService inclui o ID sob a FK na B+.
Alterar voo	Arquivo mantém a PK no Hash; a B+ remove a associação anterior e registra a nova FK.
Excluir voo	Arquivo aplica lápide e remove a PK do Hash; a B+ remove o ID da lista da FK.
Falha no fluxo	O serviço tenta reconstruir a B+ a partir dos voos ativos para restabelecer a correspondência.
Reabertura	Hash e B+ são carregados; se a conferência detectar divergência, o índice é reconstruído.

Decisões 6 e 7

A maior mudança arquitetural foi substituir o vínculo N:N legado por idCliente em Voo. Arquivo recebeu a responsabilidade de manter o Hash por PK, ArvoreBMais passou a persistir o índice de FK e AgenciaService concentra a coordenação entre os dois. Essa divisão permite demonstrar cada estrutura e reduz o acoplamento da interface com a persistência.

Persistência de arquivos. Os dados e índices ficam no diretório dados. O README do repositório descreve a compilação com JDK 21, os caminhos de cada arquivo persistido e o roteiro de demonstração. Dados de execução não são versionados; o repositório contém o código, a documentação e as instruções necessárias para recriar a base local.